Shaheed Zulfikar Ali Bhutto Institute of Science & Technology University

DEPARTMENT OF ROBOTICS & ARTIFICIAL INTELLIGENCE

Total Marks: 7.5
Obtained Marks: _____

Artificial Neural Network

(Lab)

Project Report

Last Date of Submission: 30th October 2026

Project Title : "HandTalkAI - Sign Language Detection Using CNN"

Submitted To: Mr. Hassan Mujtaba

Student Name: Amna Khalid

Registration Number: 24108112

DEPARTMENT OF ROBOTICS & ARTIFICIAL INTELLIGENCE

HandTalkAI - Sign Language Detection Using CNN

Executive summary

HandTalkAI recognizes static American Sign Language (ASL) alphabet poses from a single photo, and its final landmark model reaches 64.6% accuracy on a person it has never seen. Three neural networks were built with TensorFlow/Keras on the public Kaggle ASL Alphabet dataset: a baseline CNN, a fine-tuned MobileNetV2 CNN, and an ANN that classifies 63 normalized MediaPipe hand-landmark coordinates.

The two image models scored 99.5% and 98.1% on validation data but gave wrong predictions on photos of new people, because the dataset contains a single signer. The landmark ANN was tested leave-one-person-out on 242 photos from three signers; adding signers to its training data raised accuracy on an unseen person from 44.5% to 64.6%. The final model is served through a Flask web interface.The project covers isolated alphabet poses only, not continuous sign-language translation.

Introduction

The project asks how an ASL alphabet classifier trained on a public dataset can be evaluated fairly on photos of new people. Sign languages use hand shape, movement, facial expression and body position, so recognizing alphabet poses is much narrower than understanding a full sign-language conversation.

Why landmarks: A CNN learns hand shapes with sliding filters and pooling, but in testing the CNN models also learned the signer, skin tone and background. MediaPipe detects 21 hand landmarks (x, y, z), giving 63 numbers that describe only hand geometry and a fully connected ANN classifies them.

Aim: To develop and evaluate neural-network models that classify ASL alphabet signs, measure how well they generalize to unseen people, and demonstrate predictions through a simple web interface.

Objectives

- Prepare the Kaggle ASL Alphabet dataset: resizing, hand cropping, landmark extraction and normalization.
- Train a baseline CNN, a MobileNetV2 transfer-learning CNN and a landmark-based ANN.
- Reduce overfitting with augmentation, dropout, batch normalization and early stopping.
- Evaluate with accuracy, precision, recall, F1-score, loss curves and confusion matrices.
- Test on photos of separately collected people and compare with dataset performance.
- Build a Flask interface that shows the predicted letter and its confidence.

Scope: The image models use the 29 dataset classes (A–Z, del, nothing, space). The landmark ANN classifies the 26 letters A–Z and reports "no hand detected" when MediaPipe finds no hand. One hand
per photo is handled, and J and Z, which need movement, are only approximated by still images. This is an educational prototype, not a validated accessibility product or a replacement for interpreters.

Data and preprocessing

The project uses three data sources: the public Kaggle dataset, landmarks extracted from it, and 242 photos of three people collected for testing.

Source	Detail
Public dataset	ASL Alphabet (Kaggle, grassknoted); 87,000 training images; 29 classes
Image-model split	Baseline CNN: 80% training (69,600) / 20% validation (17,400). MobileNetV2: hand-cropped images, up to 1,000 per class
Landmark data (Kaggle)	Up to 1,000 images per letter A–Z (26,000 attempted); a hand was found in 23,737 (about 91%)
Own photos	242 phone photos from 3 people (78 / 82 / 82), letters A–Z; a hand was found in all 242; used only for testing and final training

Image models: Images are resized to 64×64 (baseline) or 128×128 (MobileNetV2), with pixel scaling inside the model. Augmentation (small rotation, zoom, translation, brightness and contrast) is applied to training data only.
Hand cropping: MediaPipe HandLandmarker finds the hand, and a padded square crop is taken. Small images are given a border and enlarged first.
Landmark features: The wrist becomes the origin and all points are divided by the largest wrist distance, which removes hand position and size and leaves 63 numbers. Augmentation uses rotation of ±20°, stretching of ±10% and small noise.

Methodology and models
The work moved from image models to landmarks after the image models failed on photos of new people.

1. Download the Kaggle ASL Alphabet dataset and verify classes and counts.
2. Train the baseline CNN: resize to 64×64, split 80% training / 20% validation, train and evaluate.
3. Detect and crop the hand with MediaPipe, resize to 128×128, and train MobileNetV2 in two phases.
4. Observe the failure on photos of new people (the domain gap).
5. Extract and normalize hand landmarks from the Kaggle images and from photos of three collected signers.
6. Train the landmark ANN, evaluate it leave-one-person-out, then train the final model on all data.
7. Serve the final model through a Flask web interface.

Model	Input	Architecture	Training
1. Baseline CNN	64×64×3	Augmentation, rescaling, 3 × (Conv2D 32/64/128 + MaxPool), Dense 128, Dropout 0.5, Softmax 29	Adam, batch 64, up to 15 epochs, early stopping
2. MobileNetV2	128×128×3 (cropped)	Augmentation, MobileNetV2 (ImageNet), global pooling, BatchNorm, Dropout 0.3, Dense 256, Dropout 0.3, Softmax 29	Phase 1: base frozen, Adam 1e-3, 8 epochs. Phase 2: top 40 layers unfrozen, Adam 1e-5, 12 epochs
3. Landmark ANN	63 numbers	Dense 256 + BatchNorm + Dropout 0.3, Dense 128 + Dropout 0.3, Dense 64, Softmax 26	Adam, batch 64, 25 epochs; each own photo plus 30 augmented copies

Evaluation: Models are evaluated with accuracy, per-class precision, recall and F1-score, confusion matrices and training curves. New-person generalization is measured with a leave-one-person-out test on the 242 own photos: the model is trained on Kaggle data plus two people and tested on the third.

Results

The image models scored above 97% on their validation split, but those scores come from a single signer and are likely optimistic.

Image model	Validation accuracy
Baseline CNN (64×64), 15 epochs	99.5%
MobileNetV2 Phase 1 (frozen base), 8 epochs	97.0%
MobileNetV2 Phase 2 (fine-tuned), 12 epochs	98.1%

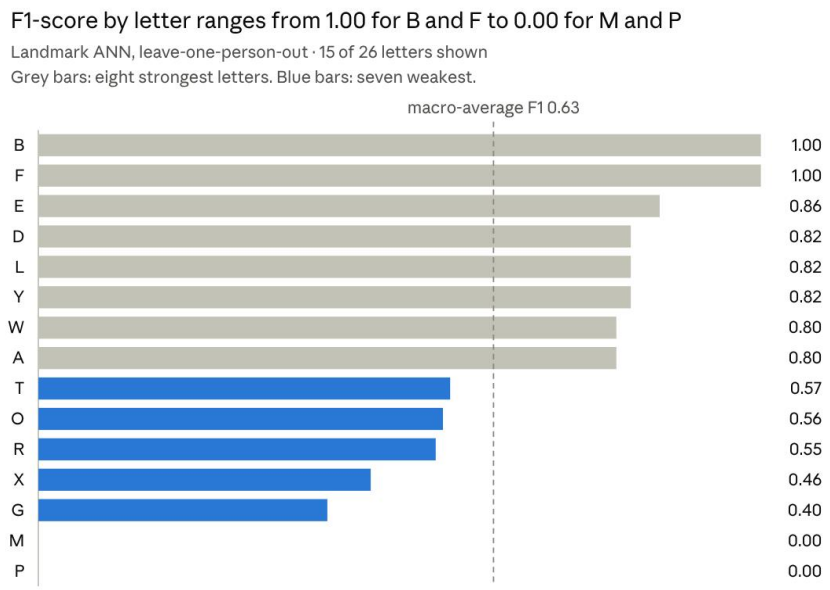
When the author's own photo was given to the image models, the predictions were wrong. Accuracy of the image models on the 242 own photos was not measured, so no percentage is claimed. This failure motivated the landmark ANN.

Landmark ANN, leave-one-person-out. The model was trained on Kaggle data plus the other two people, and tested on the held-out person. Adding signers raised average accuracy on an unseen person by about 20 percentage points.

Held-out person	Kaggle only	Kaggle + other two people
Person_1	61.5%	75.6%
Person_2	37.8%	57.3%
Person_3	34.1%	61.0%
Average	**44.5%**	**64.6%**

Over all 242 held-out predictions, accuracy was 0.64, and the macro-average precision, recall and F1-score were 0.64, 0.65 and 0.63.

Per-class results. F1-score varies widely by letter. B and F score 1.00, while M and P score 0.00.



Leave-one-person-out test on 242 own photos · 15 of 26 letters (strongest 8 and weakest 7)

Analysis and discussion

A high dataset score did not predict accuracy on new people. The Kaggle data has a single signer, so the CNN models learned the signer, skin tone and background together with the hand shape, and the 99.5% and 98.1% validation scores hid this.
Landmarks helped but did not close the gap. Passing only 63 geometry values removes background, lighting and skin tone, and training on three extra signers added about 20 points on an unseen person (44.5% to 64.6%). Person_2 and Person_3 remained the hardest to predict, at 57.3% and 61.0%.
The weakest letters (the blue bars in the Results chart) are mostly look-alike closed-hand or similar-orientation signs, where the 21 landmarks differ only slightly. The 64.6% figure comes from only three people, with about nine photos per letter each, so it is a rough estimate rather than a firm benchmark.

Web interface and tools

The landmark ANN is served by a Flask application running in Google Colab. The user uploads or captures a photo, and the server detects the hand, extracts the 63 features, and returns the predicted letter, its confidence, the top three letters and a preview of the detected points. If no hand is found, it shows a message instead. Prediction is single-image, not continuous video.

Purpose	Tool
Models	Python, TensorFlow/Keras
Hand detection and landmarks	MediaPipe
Image handling	OpenCV, NumPy
Curves and confusion matrices	Matplotlib
Metrics	Scikit-learn
Web interface	Flask
Environment and dataset	Google Colab, Kaggle

Limitations

- Only isolated alphabet poses are recognized. J and Z need motion, and look-alike letters (M, N, S, T) are confused.
- The Kaggle data has one signer, so dataset accuracy greatly exceeds accuracy on new people. No duplicate check was done.
- The 64.6% result comes from only 3 people (about 9 photos per letter each) and is a rough estimate.
- Confidence scores are not calibrated, and the interface runs inside a Colab session. It is a student prototype only.

Risks and how they were handled

Risk	Mitigation used
Overfitting	Training-only augmentation, dropout, batch normalization, early stopping
Low accuracy on new people	Landmark features, photos from extra signers, leave-one-person-out testing
Colab disconnects or lost files	Landmarks and models downloaded as backups
GPU limits	The landmark ANN and the Flask app run on a CPU
Misleading accuracy	Per-class results and a test on unseen people, reported honestly

Conclusion and Future work

HandTalkAI shows that a high dataset score alone does not prove a sign-recognition model works for other people. The CNN models reached 99.5% and 98.1% on the dataset but did not transfer to new photos. Replacing pixels with MediaPipe landmarks and adding three extra signers raised accuracy on an unseen person from 44.5% to 64.6%, a clear but incomplete improvement that points to the need for many more signers.
The project stays within static alphabet classification and does not claim to translate sign language.

Future work

- Collect photos from many more signers.
- Add a confidence threshold and optional per-user calibration.
- Move the interface to permanent hosting.
- Use landmark-sequence (LSTM) models for J, Z and continuous signing.
- Add text-to-speech output.

References

1. grassknoted, "ASL Alphabet," Kaggle dataset. kaggle.com/datasets/grassknoted/asl-alphabet (accessed October 2026).
2. Y. LeCun, L. Bottou, Y. Bengio and P. Haffner, "Gradient-based learning applied to document recognition," Proc. IEEE, vol. 86, no. 11, pp. 2278–2324, 1998.
3. M. Sandler, A. Howard, M. Zhu, A. Zhmoginov and L.-C. Chen, "MobileNetV2: Inverted residuals and linear bottlenecks," in Proc. IEEE/CVF CVPR, 2018, pp. 4510–4520.
4. F. Zhang et al., "MediaPipe Hands: On-device real-time hand tracking," arXiv:2006.10214, 2020.
5. TensorFlow/Keras, MediaPipe, OpenCV, Flask and Scikit-learn documentation (accessed October 2026).
6. F. Chollet, Deep Learning with Python, 2nd ed. Shelter Island, NY: Manning Publications, 2021.